BRIGHTLEARN TUTORIALS

DATA SCIENCE

SQL Practice Exercise 4 : Joins, UNION, Filtering and Aggregates

Table 1 : Employees
Table 2 : Projects

1. SQL Joins

1. SQL query to retrieve all employees and their assigned projects using INNER JOIN

Syntax :

```
SELECT a.EmployeeID, a.FirstName, a.LastName,
       b.ProjectName
FROM Employees AS a
INNER JOIN Projects AS b
ON a.EmployeeID = b.EmployeeID;
```

Output :

EmployeeID	FirstName	LastName	ProjectName
1	John	Doe	AI Development
1	John	Doe	Cybersecurity Audit
2	Alice	Smith	Employee Training
3	Bob	Johnson	Financial Analysis
5	Emma	Wilson	Market Expansion
6	Michael	Clark	Risk Management

2. SQL query to retrieve all employees and their assigned projects, including employees who have no projects using a LEFT JOIN

Syntax :

```
SELECT A.EmployeeID, A.FirstName, A.LastName,
    A.Department, A.Salary, B.ProjectID, B.ProjectName,
    B.Budget, B.Status
FROM Employees AS A
LEFT JOIN Projects AS B
ON A.EmployeeID = B.EmployeeID;
```

Output :

EmployeeID	FirstName	LastName	Department	Salary	Project-ID
1	John	Doe	IT	70000	101
1	John	Doe	IT	70000	103
2	Alice	Smith	HR	60000	102
3	Bob	Johnson	Finance	75000	104
4	David	Brown	IT	72000	NULL
5	Emma	Wilson	Sales	65000	105
6	Michael	Clark	Finance	80000	106

ProjectName	Budget	Status
AI Development	100 000	Completed
Cybersecurity Audit	75 000	Pending
Employee Training	50 000	Ongoing
Financial Analysis	90 000	Ongoing
NULL	NULL	NULL
Market Expansion	65 000	Completed
Risk Management	80 000	Pending

3. SQL query to retrieve all projects and their assigned employees, including projects that have no employees using Right Join

Syntax:

```
SELECT A.ProjectID, A.ProjectName, A.Budget,
  A.status, B.EmployeeID, B.FirstName, B.LastName,
  B.Department, B.Salary
FROM Employees AS B
RIGHT JOIN Projects AS A
ON B.EmployeeID = A.EmployeeID;
```

Output:

ProjectID	ProjectName	Budget	status
101	AI Development	100 000	Completed
102	Employee Training	50000	Ongoing
103	Cybersecutity Audit	75 000	Pending
104	Financial Analysis	90 000	Ongoing
105	Market Expansion	65000	Completed
106	Risk Management	80 000	Pending

EmployeeID	FirstName	LastName	Department	Salary
1	John	Doe	IT	70000
2	Alice	Smith	HR	60 000
1	John	Doe	IT	70 000
3	Bob	Johnson	Finance	75000
5	Emma	Wilson	Sales	65000
6	Michael	Clark	Finance	80 000

4. SQL query to retrieve all employees and projects including those without a match using FULL OUTER JOIN

Syntax :

```
SELECT A.EmployeeID, A.FirstName, A.LastName,
  A.Department, A.salary, B.ProjectID, B.ProjectName,
  B.Budget, B.status
FROM Employees AS A
FULL OUTER JOIN Projects AS B
ON A.EmployeeID = B.EmployeeID;
```

Output :

EmployeeID	FirstName	LastName	Department	Salary
1	John	Doe	IT	70000
1	John	Doe	IT	70000
2	Alice	Smith	HR	60000
3	Bob	Johnson	Finance	75000
4	David	Brown	IT	72000
5	Emma	Wilson	Sales	65000
6	Michael	Clark	Finance	80000

ProjectID	ProjectName	Budget	status
101	AI Development	100000	Completed
103	Cybersecurity Audit	75000	Pending
102	Employee Training	50000	Ongoing
104	Financial Analysis	90000	Ongoing
NULL	NULL	NULL	NULL
105	Market Expansion	65000	Completed
106	Risk Management	80000	Pending

2. UNION & UNION ALL

5. SQL query to retrieve all unique cities where employees are located and project status

Syntax:

```
SELECT City AS Location
FROM Employees
UNION
SELECT Status AS Location
FROM Projects;
```

Output:

Location
New York
Los Angeles
Toronto
London
Sydney
Completed
Ongoing
Pending

6. SQL query to retrieve all cities where employees are located and project statuses allowing duplicates

Syntax:

```
SELECT City AS Location
FROM Employees
UNION ALL
SELECT Status AS Location
FROM Projects;
```

Output : Location

New York

Los Angeles

Toronto

London

Sydney

New York

Completed

Ongoing

Pending

Ongoing

Completed

Pending

3. Filtering Statements

1. SQL query to retrieve who earns more than 70 000

Syntax :

```
SELECT EmployeeID, FirstName, LastName,
Department, salary
FROM Employees
WHERE Salary > 70000;
```

Output :

EmployeeID	FirstName	LastName	Department	salary
3	Bob	Johnson	Finance	75 000
4	David	Brown	IT	72 000
6	Michael	clark	Finance	80 000

8. SQL query to retrieve employees working in either IT or Finance departments

Syntax :

```
SELECT EmployeeID, FirstName, LastName,
    Department, Salary
FROM Employees
WHERE Department IN ('IT', 'Finance');
```

Output :

EmployeeID	FirstName	LastName	Department	Salary
1	John	Doe	IT	70000
3	Bob	Johnson	Finance	75000
4	David	Brown	IT	72000
6	Michael	Clark	Finance	80000

9. SQL query to retrieve projects not yet completed

Syntax :

```
SELECT ProjectID, ProjectName, Budget, status
FROM Projects
WHERE Status != 'Completed';
```

Output :

ProjectID	ProjectName	Budget	Status
102	Employee Training	50000	Ongoing
103	Cybersecurity Audit	75000	Pending
104	Financial Analysis	90000	Ongoing
106	Risk Management	80000	Pending

10. SQL query to retrieve projects with budgets greater than 70 000 and not completed

Syntax :

```
SELECT ProjectID, ProjectName, Budget, status
FROM Projects
WHERE Budget > 70000
AND Status != 'Completed';
```

Output :

ProjectID	ProjectName	Budget	Status
103	Cybersecurity Audit	75000	Pending
104	Financial Analysis	90000	Ongoing
106	Risk Management	80000	Pending

11. SQL query to retrieve employees from New York or Toronto, ordered by salary in descending order

Syntax :

```
SELECT EmployeeID, FirstName, LastName,
 Department, Salary, city
FROM Employees
WHERE City IN ('New York', 'Toronto')
ORDER BY salary DESC;
```

Output :

EmployeeID	FirstName	LastName	Department	Salary	City
6	Michael	Clark	Finance	80000	New York
3	Bob	Johnson	Finance	75 000	Toronto
1	John	Doe	IT	70 000	New York

12. SQL query to retrieve top 3 highest paid employees

Syntax :

```
SELECT EmployeeID, FirstName, LastName,
Department, Salary
From Employees
ORDER BY Salary DESC
LIMIT 3;
```

Output :

EmployeeID	FirstName	LastName	Department	Salary
6	Michael	Clark	Finance	80000
3	Bob	Johnson	Finance	75000
4	David	Brown	IT	72000

4. Aggregate Functions with GROUP BY & HAVING

13. SQL query to find total salary per department sorted in descending order

Syntax :

```
SELECT Department, sum(salary) AS TotalSalary
FROM Employees
GROUP BY Department
ORDER BY TotalSalary DESC;
```

Output :

Department	TotalSalary
Finance	155 000
IT	142 000
sales	65 000
HR	60 000

14. SQL query to find average salary per city, but only include cities where average salary is greater than 65000

Syntax :

```
SELECT city, AVG(salary) AS AverageSalary
FROM Employees
GROUP BY city
HAVING AVG(salary) > 65000;
```

Output :

City	AverageSalary
New York	75000
Toronto	75000
London	72000

15. SQL query to count number of employees per department, including departments with more than 1 employee

Syntax :

```
SELECT Department, COUNT (EmployeeID) AS
    EmployeeCount
FROM Employees
GROUP BY Department
HAVING COUNT (EmployeeID) > 1;
```

Output :

Department	EmployeeCount
IT	2
Finance	2

16. SQL query to retrieve number of projects per status, but only statuses at least 2 projects

Syntax :

```
SELECT Status, COUNT (ProjectID) AS ProjectCount
FROM Projects
GROUP BY Status
HAVING COUNT (ProjectID) >= 2;
```

Output :

Status	ProjectCount
Completed	2
Ongoing	2
Pending	2

17. SQL query to retrieve the total project budget per employee, but only for employees who are managing projects worth more than 150 000

Syntax :

```
SELECT A.EmployeeID, A. FirstName, A. LastName,
  SUM (B. Budget) AS TotalProjectBudget
FROM Employees AS A
JOIN Projects AS B
ON A. EmployeeID = B. EmployeeID
GROUP BY A. EmployeeID, A.Firstname, A.Lastname
HAVING SUM (B.Budget) > 150000;
```

Output:

EmployeeID	FirstName	LastName	TotalProjectBudget
1	John	Doe	175000